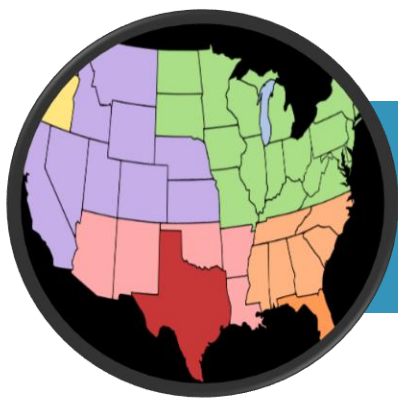




Oregon's Fall - The Final Stand

GAME ENGINEERING | CAPSTONE

STUDENT TEAM

<Ryan Barkman>, 4th Year Student

E: barkmanr@sheridancollege.ca

<Aidan Skomra>, 4th Year Student

E: skomraa@sheridancollege.ca

<Mark Seferiades>, 4th Year
Student

E: seferiam@sheridancollege.ca

SUPERVISOR

Prof. Rogerio de Leon Pereira

E:

rogerio.deleonpereira@sheridancollege.ca

T: <office phone # and ext>
Sheridan College

ABSTRACT

This document serves as the report for the game engineering capstone project titled Oregon's Fall - The Final Stand, a 2D single-player zombie survival game with strong modding functionality. Research on the gaming industry has led to a conclusion that games with better modding capabilities have higher longevity in terms of player count and player engagement compared to similar games without. This capstone project aims to incorporate cross-platform, easy to use modding to garner those benefits. The project's team utilizes the Agile approach of software development, aided by using JIRA. The release plan and risk management analysis signifies the suitability of the project. Visual Paradigm was used to articulate the project requirements and architecture. An iteration plan is formulated from the project's inception to the final product. The team has settled upon a layered architecture, with the modding and cloud systems necessitating higher-level infrastructure logic. The game will be developed on Unity, which complements the layered architecture and provides useful features for the testing strategy the team has proposed.

ABOUT CAPSTONE PROJECTS

TIMELINES • PROGRAM • SCHOOL

- **Sept 2024 – December 2024:** [Capstone Project Inception](#), 4-credit course (18 hours / week)
- **September 2025 – December 2025:** Capstone Project, 4-credit course (18 hours / week)

PROGRAM • SCHOOL

- [Hons. Bachelor of Appl. Computer Science \(Game Engineering\)](#)
- [Applied Computing, Faculty of Applied Science and Technology](#)

Table of Contents

Sheridan

Introduction.....	4
Project Overview	4
Domain and Industry Overview.....	6
Problem Description.....	6
Solution Description	7
Cloud Computing.....	8
Advanced Areas of Computer Science	8
Solution Impact	8
Solution Feasibility.....	8
Design and Construction.....	9
Deployment.....	9
Adoption	9
Project Management	9
Management Process	9
Responsibility Matrix	9
Release Plan	10
Risk Management.....	11
Project Requirements	12
System Context	12
Use-Cases	13
User interface.....	14
Project Architecture.....	20
Architecture Overview.....	20
System Components.....	21
Deployment Model.....	22
Project Plan.....	23
Iteration Plan.....	23
Risk Management Plan	23
Validation and Testing	24
Testing Strategy.....	24

Validation Results 25

Conclusion25

 Project Suitability 25

 Domain Expert Evaluation..... 25

 User Testimonials 26

 Future Work 26

Glossary26

Bibliography.....Error! Bookmark not defined.

INTRODUCTION

This report provides an overview of the Oregon's Fall – The Final Stand capstone project and the research areas supporting its development. It consists of the following sections:

- Project Overview – a high-level summary of the capstone project concept.
 - Domain and Industry Review – a definition of the project's industry, according to the Government of Canada, along with key statistics relevant to that industry.
 - Problem Description – an outline of the game-industry problem this capstone intends to address.
 - Solution Description – an explanation of how the capstone project proposes to solve the identified problem.
 - Solution Impact & Feasibility – an assessment of the solution's potential effectiveness and its practicality from a development standpoint.
 - Cloud Computing – An overview of how cloud technologies (AWS S3) are used for mod storage and distribution
 - Advanced Areas of Computer Science – Summary of advanced components of this project
 - Deployment – An overview of deployment platforms we will use
 - Adoption – A review of what factors will make users play our game
- Project Management – Team Roles, Tools, Development Process
 - Management Process – Description of the SCRUM workflow
 - Responsibility Matrix – Responsibilities for each group member
 - Release Plan – A summary of project's releases
 - Risk Management – Major risks that have been identified and strategies to combat them
- Project Requirements – Overview of system requirements
 - System Context – Description of actors in the system
 - Use-Cases – A summary of the major use cases
 - User Interface – Wireframes and UI from the game
- Project Architecture – Description of overall architecture
 - Architecture Overview – Explanation of the system structure
 - System Components – Details of components and their responsibilities
 - Deployment Model – A view of how the system is deployed
- Validation and Testing – Summary of test strategies and results
 - Testing Strategy – Approach for system validation and testing
 - Validation Results – Outcomes of the implemented tests

PROJECT OVERVIEW

Oregon's Fall - The Final Stand is a 2D single-player zombie survival game concept created by Ryan Barkman, Aidan Skomra, and Mark Seferiades. It draws inspiration from *The Last Stand* franchise [6] and the *Oregon Trail* remake [7]. The objective is to travel across the United States, starting in Oregon and journeying east in an attempt to escape in one of 3 destinations:

- Canadian maritime islands from Maine
- The Caribbean Islands off the coast of Florida

- Mexico via bottom of Texas

Core Gameplay Elements:

- **Traveling** - Includes both overworld map travel and inner-city navigation.
- **Looting** - Players explore city buildings for essential supplies.
- **Encounters** - Random events during travel, which can be beneficial or dangerous.
- **Zones** - Different regions on the map offering unique pools of encounters and events.
- **Combat** - Hostile engagements triggered through specific encounters.
- **Survivors** - Recruitable NPCs who assist in survival and combat.
- **Survival** - players must manage combat effectiveness and resource scarcity.

Modding will be a major part of the game. To enable this, an in-game mod system will be included, offering tools and templates that allows creating and using content accessible across different platforms. Since tradition modding solutions are largely limited to PC such as steam, other gaming platforms don't provide the same level of mod support like PlayStation or Xbox. Our goal is to ensure that at least PlayStation users can access the in-game modding features, though this may change depending on development constraints. All mods will be stored with Amazon S3 cloud services that will be grabbed via unity calls.

Oregon's Fall will play like a roguelike. Each journey is a standalone run, where death sends the player back to the start in a new game. With every attempt, players can learn from their mistakes and aim to get farther than their previous attempt.

Team Roles:

- **Ryan Barkman** - *Project Owner*, responsible for aligning the game with the overall vision and goals.
- **Mark Seferiades** - *SCRUM Master*, ensuring the team stays on track and deadlines are met.
- **Aidan Skomra** - *Risk Analyst*, evaluating the feasibility and impact of features, and ensuring they function as intended.

Inspirations:

To better understand core gameplay mechanics and design philosophy, we recommend looking into the following video games:

- **The Oregon Trail remake** by Gameloft – for its travel and decision-making features [7]
 - https://store.steampowered.com/app/2013360/The_Oregon_Trail/
 - Steam website
- **The Last Stand: Union City** by Con Artist Games – for its combat and looting features [4]
 - <https://armorgames.com/play/12009/the-last-stand-union-city>
 - Original flash location
- **The Last Stand 2** by Con Artist Games– for its city-to-city travel [5]
 - <https://armorgames.com/play/12009/the-last-stand-union-city>
 - Original flash location
- **Project Zomboid** by The Indie Stone– for the benchmark of the zombie survival genre through their gameplay and mods [8]
 - https://store.steampowered.com/app/108600/Project_Zomboid/

- Steam website

DOMAIN AND INDUSTRY OVERVIEW

The video game industry is one of the largest and most influential in the entertainment field in the world. In the United States alone, *the Entertainment Software Association (ESA)* reports that 64% of Americans; over 205 million people play video games, with consumers spending around \$59 billion on gaming products each year [9]. Gaming is especially common among younger audiences, with 83% of American children playing games regularly. This is relevant to Canada as well, as Canadian consumption patterns are similar to the United States especially with digital content, so gaming trends in the U.S. typically scale proportionally to Canada's population. According to *Made in CA* 61% of Canadians play video games [13].

On a global scale, Statista estimates that 2.7 billion people of the world's population play video games [11]. Other sources, such as Exploding Topics, place the global number even higher at 3.3 billion players and value the worldwide gaming market at \$522 billion [12]. In terms of age demographics, 40% fall into the 18-34 category. This group is particularly important for our project since they lived through the "Flash game era," which heavily influences the style and nostalgia behind our game. Additionally, 63% of gamers identify as casual players, making up the majority of the global gaming audience, approximately 2 billion people who play regularly; these are also the group more likely play a video game like ours as they are more likely have access to a platform that can play these types of games.

Engagement with gaming culture also continues to rise. For example, 154 million people watched the 2024 Game Awards (GameSpot) [10], demonstrating that gaming events attract mainstream attention. Popular gaming platforms are consistently active as well; according to SteamDB, around 32 million users are online on Steam at any given time [3].

Overall, these statistics highlight that the gaming industry is massive, and a major player in modern culture. Although the market spans thousands of different IPs and genres, the sheer size of the audience shows that gaming is an industry worth exploring since it affects most of the population.

PROBLEM DESCRIPTION

- **Mod Access Across Platforms:** Mod accessibility varies significantly across gaming platforms. Platforms other than PC generally have limited mod support. Consoles like PlayStation and Xbox typically offer the same content for major titles, whereas platforms like Steam provide a robust modding system that can significantly enhance games. As a result, players on PlayStation or Xbox often have access to less content compared to Steam users. This can be seen in *Black Ops 3* [14], where the content on PlayStation is good, but is nowhere close compared to on Steam which has almost endless mods at their disposal backed by the community.

Modding has been shown to substantially boost a game's player base; for example, according to SteamDB charts, *Black Ops 3* ranks 2nd (or 3rd) in player count among all Call of Duty titles, only behind the Call of Duty Launcher, which hosts *Black Ops 6* and *Black Ops 7* (the newest ones) [3]. This sustained popularity is largely attributed to *Black Ops 3*'s Zombie modding community, which benefits from having a Steam Workshop; the only Call of Duty title with Steam mod support.

- **Game Longevity:** Another major challenge with video games is maintaining long-term player engagement. Publishers often aim to keep users invested in their games to drive ongoing revenue through additional content or to ensure that newer titles benefit from the popularity of previous releases. However, sustaining player interest is difficult, particularly for single-player games, which tend to have a finite lifespan. Once players complete the story, many stop playing entirely.

Some developers attempt to extend playtime through achievements or collectibles, but these mostly appeal to completionists, who represent only a small fraction of the overall player base. Achievements are random tasks that publishers create that gives players little kudos for completing them where usually there is one for getting all of them. A way to say that a player completed a game 100%; this doesn't keep a majority of players.

Online games face similar challenges. Even popular multiplayer titles often experience a sharp decline in player base within a year as gamers move on to newer releases. For example, annual franchises like Call of Duty tend to push players from one installment to the next, but this strategy does not necessarily create lasting engagement with the individual games themselves.

Adding new official content is one of the most effective methods of sustaining engagement, particularly for multiplayer games. However, content creation requires significant time, resources, and personnel. For smaller development teams (Like us), continuously producing new content at a large enough to keep fans pleased.

Certain genres naturally promote replayability. Roguelikes, for instance, offer variability and procedural content that can keep players engaged longer than traditional single-player games. However, even in roguelikes, players may stop after some successful runs, leaving primarily the completionists or dedicated fans engaged who are not a part of the majority of the players. Longer games can also extend engagement, but they require bigger budgets, bigger teams, and more time which at our scale is impossible. It can also lead to flops where trends change after long periods of time.

SOLUTION DESCRIPTION

To address these challenges, we propose implementing an in-game modding system to support long-term player engagement across multiple platforms. This system will allow user generated content to be created, shared, and accessed directly within the game.

Mods will be stored and managed through a cloud-based database, enabling compatibility between PC and alternative platform formats. By centralizing mod distribution, the system ensures access to community-created content regardless of platform limitations.

Since we are incorporating a modding system, we don't need a big team to add as much content as possible. The game still needs to be enjoyable to generate a community to add content.

Cloud Computing

We are using Amazon S3 cloud storage to store mod data

Advanced Areas of Computer Science

- In game mod system.
 - Asset Bundles with integrated code implementation
- Entity AI
- High-volume Runtime Data Persistence
 - Saving and loading large volumes of dynamically generated runtime data, including reconstructing scenes that are not present by default

SOLUTION IMPACT

In the best-case scenario, if our modding system is robust enough, it could be reused across other games. This would allow developers to include mod support more easily and potentially bring mod access to console gaming.

SOLUTION FEASIBILITY

This project is feasible due to the 2D scope and simple mechanics. Core Systems are straightforward like travel and combat. While unity's SDK's support modding and cloud saving.

Most of the project's aspects were covered in courses in the game engineering discipline that all team members have taken, including gameplay programming for both 2D games and 3D games in Unity, game AI development in Unity, and basic modding functionality with scripting. Core courses as part of the Computer Science program have also been taken by all members, providing knowledge in best programming practices, cloud integration and usage, and the software engineering practice.

Risks: Mod Security, Small Team Size, Time

Mitigation: Mod Validation, Clear Workload Division

DESIGN AND CONSTRUCTION

Deployment

Feasible through platforms like Steam, PlayStation, Itch.io. Uses a built saving system to ensure reliable saving. Products like Amazon Cloud allow saving mod data online and accessible anywhere with an internet connection.

Adoption

Feasible if supported by nostalgia appeal and modding community engagement. Risks include crowded indie markets and limited marketing. Adoption is strengthened by low costs, replay ability, and strong community-driven content.

PROJECT MANAGEMENT

MANAGEMENT PROCESS

Our team uses the Agile Scrum process in Jira. The PMP (Scrum Board) manages features, stories, tasks, and subtasks. The RMP (Kanban Board) tracks potential risks. Our work is organized into sprints or iterations that last 1 week, our features are implemented, tested, and reviewed. This process allows us to work together as a team and deliver our project on time, with all the required features.

RESPONSIBILITY MATRIX

Project Responsibilities	Ryan Barkman	Mark Seferiades	Aidan Skomra
Project Management			
Project Owner	Ryan		
SCRUM MASTER	Ryan	Mark	
Risk Analyst			Aidan
Requirement Engineering			
Requirements / Business Analyst	Ryan		
Stakeholder Champion (by Stakeholder)	Gamers	Modders	Content Creators
Functional Area Champion (by functional area)	Save-Data	Modding	Cloud Mod-Database

User Experience Design Lead	Ryan		
Software Architecture			
Software Architect		Mark	
Requirements Model Lead	Ryan		
Domain Model Lead	Ryan		
Design Model Lead		Mark	
Deployment Model Lead		Mark	
Interaction Model Lead		Mark	
Construction			
Full-Stack Developer (UI, code and unit-tests)	Travel	Combat	Player and Looting
Integration / DevOps Lead			Aidan
Testing			
QA Lead			Aidan
Verification & Validation Champion (by Functional Area)			Aidan
Test Model Lead	Ryan		
Support			
Tool and Devices Support			Aidan
Communication Support	Ryan		

RELEASE PLAN

- Inception Release:

Find all requirements that the game will try to meet. Plan all features that we want to implement for the Alpha. Plan roles for each member to be responsible of.

- **Architecture Release:**
Determine all resources to be used in the project and research to make sure each member knows how to use the services that we will be using. Create diagrams in Visual Paradigm for the design and architecture of the project's systems
- **Elaboration Release:**
Refine requirements and plan risk analysis. Making sure that we are on task when working on the Alpha release. Make prototypes of useable code.
- **Alpha Release:**
Working functionality of basic game systems. Some features should be worked on early to identify possible risks in the future. No fancy looks, just basic functionality.
- **Beta Release:**
More functionality of less important features for improved gameplay. More effort into look of UI and other features to make game look smoother. Implement modding tools, and ensure basic areas are moddable. Test all features for final release.
- **Final Release:**
Focus goes into UI and other looks. Fix all issues found in testing. Fully Useable.

RISK MANAGEMENT

Top 5 Risks:

- 1) Cloud Save Downtime
Problem: Unity Cloud or provider may go offline during gameplay, blocking persistence.
Strategy: Implement local save fallback. Notify players if cloud is unavailable, research alternate providers like AWS.
Prob: High
Impact: Serious
Risk Area: Project
- 2) Failure to implement cloud-based mod services
Problem: Members of the development team aren't well versed in cloud based and have already provided mod addition to game development, so a higher level of understanding is required in this field.
Strategy: limit gameplay feature to give more time to this. This is part of the advanced area of computer science which means that this is on of the most important feature of the project. Since Mark is the SCRUM master, he must decide if we need to switch are focus into Mods.
Prob: Moderate
Impact: Serious
Risk Area: Product
- 3) Teamwork management
Problem: We need to be able to communicate with each other to turn out a good product.

Strategy: Try to plan more meetings, spending more work trying to understand. Give the person in charge of that specific role the chance to put their boot down to force people into a single view if time is becoming an issue (not personal, business).

Prob: High

Impact: Serious

Risk Area: Business

4) Insufficient Level of Moddability

Problem: Various areas of the game may prove too difficult to make moddable through game mods. This not only goes against our original vision for most of the game to be moddable, but it may also cause discontent with our stakeholders (gamers and modders).

Strategy: Need to assess and focus on the most important elements of the game to make moddable, prioritizing them over other features. Then, we scale back, when necessary, if the selected areas prove too difficult.

Prob: Medium

Impact: Tolerable

Risk Area: Product

5) Amount of Game Code

Problem: This Project is viewed as a manageable difficulty, but the number of requirements could prove to be time-consuming for a 3-person team.

Strategy: Work on Important features first, switch to simplified version of features later to simplify goals to complete more goals. Change features back to original values if goals gave time to.

Prob: Moderate

Impact: Tolerable

Risk Area: Project

PROJECT REQUIREMENTS

This section details the project's requirements. It identifies the actors a part of the system's context, and highlights the use cases of the various functional areas, linking them to their respective diagrams on Visual paradigm.

SYSTEM CONTEXT

Actors:

Player <<primary>>:

These are the users who actively engage with the game. Our goal is to ensure they enjoy the gameplay experience and to track instances where they encounter frustration or dissatisfaction. By addressing these issues, we can keep players engaged and attract new participants as the community thrives.

Modders <<primary>>:

These are users who create and contribute additional content to the game. Mods can be downloaded by other players and are integrated into the system, where they are stored securely in the cloud for easy access and management. We want to ensure that our system meets their needs of adding content.

System Context Diagram: LastStand.vpp://diagram/9Mr6L5mGAqAEzRTs

USE-CASES

Save System: (LastStand.vpp://diagram/rFNrkVmGAqAEzWID)

- SaveData
- LoadData
- DeleteData
- SaveLevelData
- LoadLevelData
- DeleteLevelData
- DeleteLevel

Travel System: (LastStand.vpp://diagram/uMXM9ImGAqAEzRCL)

- Pick-A-Point
- NavMesh
- NavMeshBool
- PlayerMarker
- Movement
- TravelState
- EncounterCheck
- Zones
- TargetLocation
- Encounter Zones
- Path
- HazardZones
- **State Travel**
 - Country Map
 - Goals
 - Cities
 - Path-ways
 - US states
- **City Travel**
 - City Maps
 - Roads
 - Buildings
 - ExitLocations

Encounter System: (LastStand.vpp://diagram/2pkgUVmGAqAEzeNb)

- Win Condition

- Spawning
- Encounter Pool
 - Generic Pool
 - State Pool
 - City Pool
 - Zone Pool
- Escaping
- Re-Entry

Cloud System:

- Delete File (LastStand.vpp://diagram/OXMlcVmGAqACaCes)
- Download File (LastStand.vpp://diagram/OLg_flmGAqACaTt.)
- Upload File (LastStand.vpp://diagram/cv7XflmGAqACaTLw)

Inventory System:

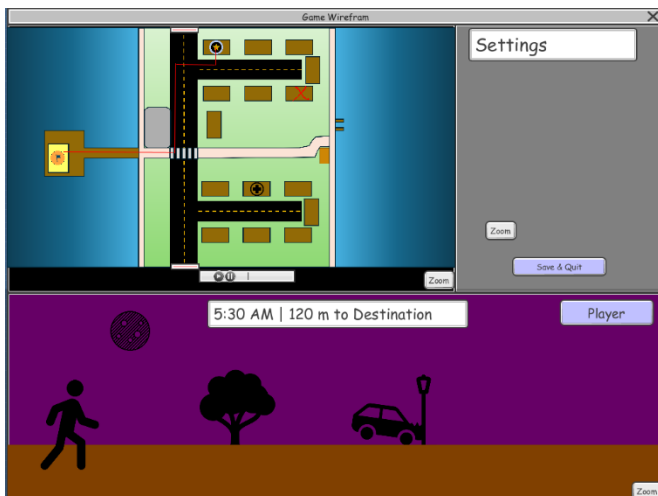
- Add Item (LastStand.vpp://diagram/wMkdcVmGAqACaCsC)
- Remove Item (LastStand.vpp://diagram/e.pdcVmGAqACaCtS)
- Use Item (LastStand.vpp://diagram/.0vDcVmGAqACaCuT)

Combat System:

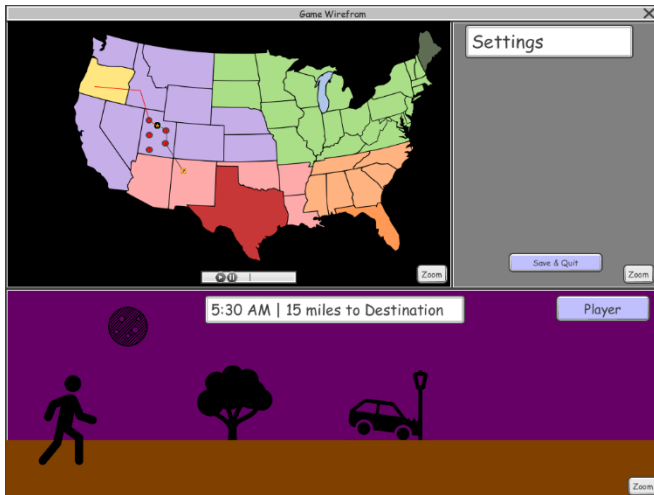
- Melee Weapon Usage (LastStand.vpp://modelelement/f..ZKNmGAqAEFy7m)
- Ranged Weapon Usage (LastStand.vpp://modelelement/_CaZKNmGAqAEFy69)
- Movement (LastStand.vpp://modelelement/tzgnqNmGAqAEFwve)

USER INTERFACE

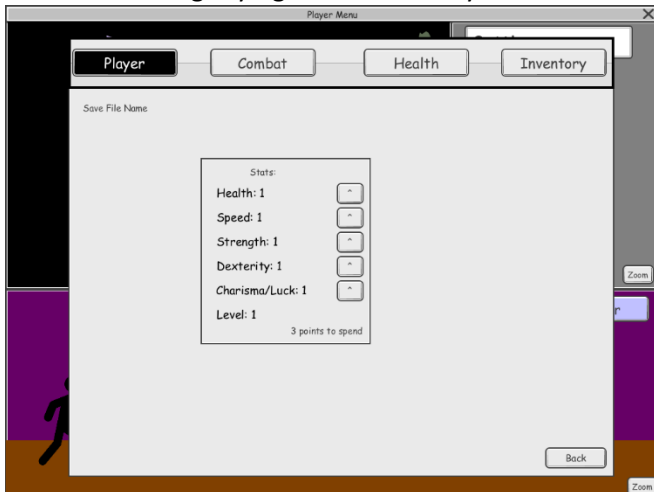
Wireframes:



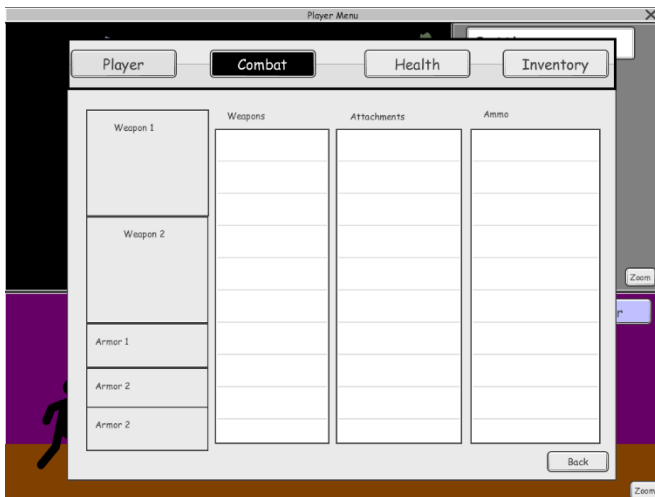
Wireframe of game scene in Travel State mode. Top Section includes a country map with different zones represented by different colors. Includes a player marker (star in this example) with a path towards the target destination formed by the NavMesh system utilizing shortcuts between cities. The red line is a path taken so far but this is a less important feature. Red circles are towns players can enter to gain loot. Also include buttons that pause and start moving again towards target. Bottom section is game scene where combat, looting and encounters happen.



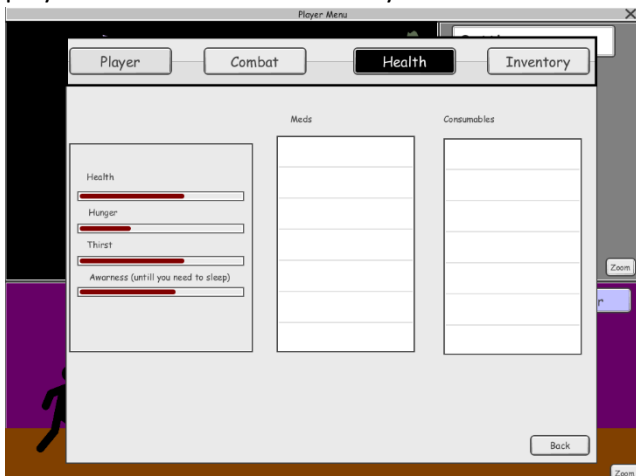
When players enter a city the travel state mode turns to city travel mode which turns the map to a city map. Most features stay the same, but players may now loot buildings. Some buildings have symbols, but they are off by default and can be shown via specific encounters. When player enters a building, it switches to loot mode. Exit locations at ends of roads signifying exit back to city travel mode



Player button visible in previous screenshots opens this tab. Tabs on top switch to different sections in the player tab. Default is Player tab where holds player stats that can be upgraded when leveled up.



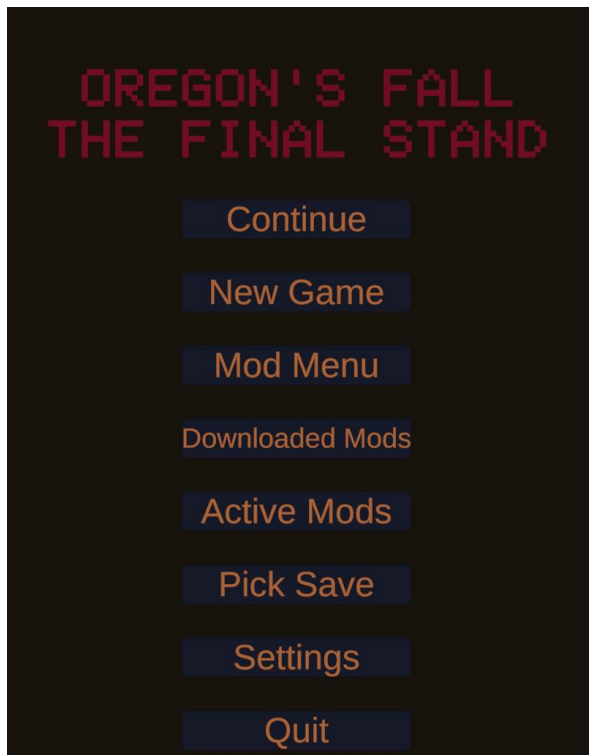
Combat section includes all weapons, attachments, and ammo types. Also has 2 slots for 2 equipped weapons that player can switch between easily. Includes an armor section that is still in discussion.



Health tab is where medicine and consumables are located. *Also includes bars representing what status the player is in. If a bar emptied the system will auto take it from inventory, not health. Items can also be clicked on to use. When health empties you die, when hunger empties you eat automatically, when thirst empties, you drink, and when awareness empties, the player sleeps for a bit (not during encounters).*

PSPI's:

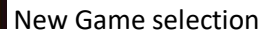
So far, we have a working mod menu with the save system hooked up. Users can so far create levels, pick created levels, continue game, and look at the template mod and cloud features. Save System saves global data like active level but also can save current level data in their own folder.

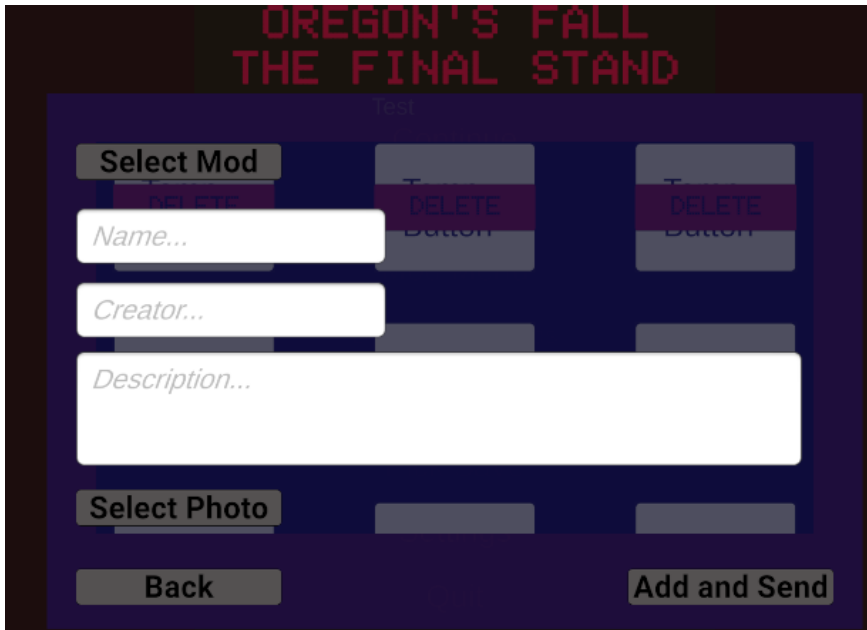


Main Menu

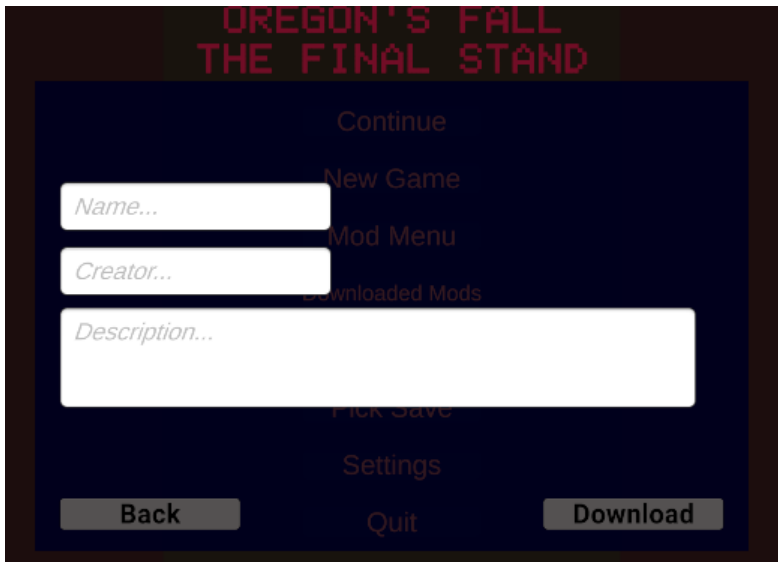


List of save files





Create Mod Template



Specific Mod Template

```

// Unity Script & references
public class SaveSystem : MonoBehaviour
{
    //activelevel.bin
    //levellist.bin
    2 references
    public static void SaveData<T>(T _data, string _path)...

    1 reference
    private static void CheckFoulder(string path)...

    1 reference
    public static void SaveLevelData<T>(T _data, string _path)...

    6 references
    public static T LoadData<T>(string _path)...

    1 reference
    public static T LoadLevelData<T>(string _path)...

    0 references
    public static void DestroyData(string _path)...

    0 references
    public static void DestroyLevelData(string _path)...

    1 reference
    public static void DestroyLevel(string _dir)...
}
    
```

Fully functioning SaveSystem

PROJECT ARCHITECTURE

All related diagrams can be viewed here -> <https://online.visual-paradigm.com/w/sbrtotdg/drive/#proj=0&dashboard>

ARCHITECTURE OVERVIEW

The system being developed is a Game System that includes both a Mod System and a Cloud System. The Game System manages the gameplay, while the Mod System and Cloud System handle mod functionality and their cloud storage, allowing users to access them. The platforms for reference are the *Oregon Trail* remake and *The Last Stand* franchise from the Flash era, as these are the primary inspirations for our own system. Knowledge of these titles helps us identify the features we want to implement. Steam is also a useful point of reference for the mod-related aspects, as we can implement similar functionality to liked features.

The system will use a layered architecture. At the base is the Save System, which allows game components to save and load data as needed. This layer interacts directly with the user's file system via `Application.persistentDataPath`.

Above it are the Mod System and the Cloud System, which function as higher-level infrastructure logic. With the Mod System positioned above the Cloud System, it interacts with the Cloud System to save and load mods, while the Cloud System interacts directly with cloud bucket storage to store individual mods and their associated data.

The Mod System also communicates with the Save System to store downloaded mods to the game itself. Above these is the game-script layer, which acts as the application-logic layer. This layer communicates with both the Save System and the Mod System to handle game-specific requirements and calculations. It also contains the various gameplay subsystems: travel, encounter, combat, item, and player; each responsible for different functional areas.

These subsystems communicate with lower systems as needed, with the travel system acting as the top-level as travel drives the game with combat as a byproduct of travelling.

The final layer is the UI layer, which converts user input into actions passed to the game-script layer.

SYSTEM COMPONENTS

Unity:

Save System:

The lowest layer of the system, responsible for handling and storing user's game data in Application.persistentDataPath on the user's device. It communicates with the application logic to save and load data directly from the file system. (*LastStand.vpp://modelelement/Uw7dClmGAqAEzRfI*)

Cloud System:

Interacts directly with Amazon S3 buckets to upload and retrieve mods. It is used by the Mod System to fulfill its requests. (*LastStand.vpp://modelelement/qLTdClmGAqAEzRfc*)

Mod System:

Communicates with both the Cloud System and Unity scripts to download and run in-game mods. It must also support importing user-provided files to send to the Cloud System. (*LastStand.vpp://modelelement/H1ddClmGAqAEzRfU*)

Travel System:

The top layer of the game script architecture that drives travel functionality. Includes two subsystems: state travel and city travel. This interacts with the Encounter System to generate combat scenarios or Item System when entering a building for loot mode. This is all handled in Travel scene on the top half of gameplay. (*LastStand.vpp://modelelement/LqFdClmGAqAEzRfD*)

Encounter System:

Spawns' encounters into game scene, which can connect to the Combat System or Item System to generate related entities from predefined random events. (*LastStand.vpp://modelelement/9XLAUVmGAqAEzeA_*)

Combat System:

Handles player movement, shooting, human AI, and zombie AI. Interacts with the Item System for potential loot drops. This is all handle in game scene on the bottom section of the game. (*LastStand.vpp://modelelement/3t3dClmGAqAEzRfv*)

Item System:

Manages looting and items. Interacts with the Player System to provide items.

Player System:

Manages stats and inventory. Forms the base of the game script hierarchy, supporting systems above it by providing statistics or items. (*LastStand.vpp://modelelement/cpA9CImGAqAEzRf3*)

External:

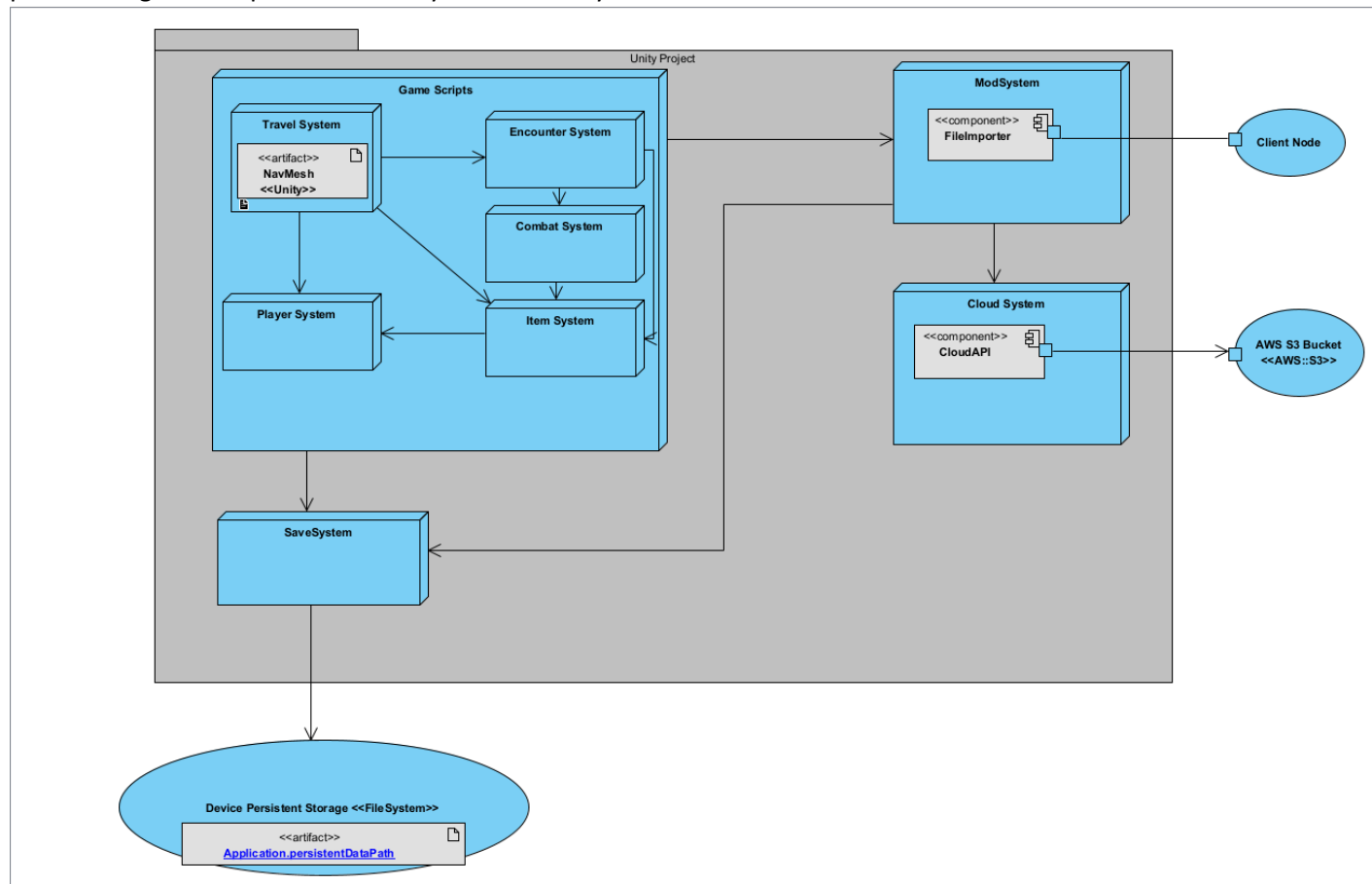
Application.persistentDataPath: Location of saved data object.

Aws s3 buckets: Cloud storage for mods.

Client Node: Clients file system to be used when importing files.

DEPLOYMENT MODEL

Unity game system with access to cloud storage containers run on a client's pc. Used in a layered approach with UI elements on the top interacting with game scripts and a mod system with them interaction with a local save system and a cloud save system. The bottom layers talk to the external resources, not part of the unity scene. Travel system, part of the game scripts include unity's NavMesh system that will be used.



LastStand.vpp://diagram/GHQycVmGAqAEzVRE

PROJECT PLAN

This section is the outline for our approach to manage the development of our game Oregon's Fall – Final Stand. It describes the overall project plan which entails iteration planning and risk management.

ITERATION PLAN

Development is split into short and organized iterations to deliver incremental functionality and validate our progress. Each of the iterations has an objective to contribute to the final release.

- **Iteration 1 – Inception**
 - Identify requirements and core gameplay features
 - Assign roles to each team member
 - Produce initial backlog of tasks in JIRA
- **Iteration 2 – Architecture**
 - Conduct research on required technologies
 - Design system architecture and other diagrams in Visual Paradigm
 - Design the structure of the Save, Mod, and Cloud systems
- **Iteration 3 – Elaboration**
 - Complete basic UI
 - Build prototypes of the core systems
 - Build Cloud and Mod systems functionality
 - Prepare for Defense Presentation
- **Iteration 4 – Alpha**
 - Develop core gameplay systems
 - Ensure travel, encounters, combat, and player systems work and have no issues
 - Implement basic modding capabilities into the game
 - Maintain focus on functionality instead of visuals
- **Iteration 5 – Beta**
 - Work on less important features and improve user experience
 - Improve UI for useability
 - Finalize modding systems
 - Conduct tests on all systems
- **Iteration 6 – Final**
 - Complete UI and all gameplay systems
 - Resolve any issues from tests
 - Prepare for final presentation

RISK MANAGEMENT PLAN

Risk management is continuously monitored throughout this project. The team has a risk board to track and mitigate risks related to all aspects of our game.

Process Overview

- **Identification**
 - Risks are recognized and recorded as soon as they come up during research
- **Analysis**
 - Each risk is evaluated by many factors to determine its severity and priority
- **Planning**
 - Mitigation strategies are developed to reduce the chance of it happening or lessen its impact
- **Monitoring**
 - All the risks are reviewed in meetings to track changes and see how well the mitigation strategy works
- **Active**
 - A risk becomes active when it starts to affect our development.
- **Resolved**
 - A risk is resolved when our mitigation strategies succeed or when the issue isn't relevant anymore

VALIDATION AND TESTING

Provide links to issues repository in BitBucket.

Results of Testing Game Features, their outcomes, and plan on moving forward.

TESTING STRATEGY

Unity Testing: Record Attempt and expected result and actual result. Works if results match.

Save System	Description	Expected Result	Result
1	SaveData<T>(T data, sting path)	Should Save Serializable Data in Application.PersistantDataPath in .bin file	Saved in .bin File
2	SaveLevelData<T>(T data, string path)	Should Save Serializable Data in a folder of Level name in Application.PersistantDataPath in .bin file. If foulder doesn't exist; create one.	Saved .bin File in LevelName folder and creates one if doesn't exist
3	T LoadData<T>(string _path)	Return Serializable Obj from Apliaction.PersistantDataPath. Return null if none exist.	Returns obj if .bin file exists and null if not.
4	T LoadLevelData<T>	Return Serializable Obj from folder of level name in Apliaction.PersistantDataPath. Returns null if none exist.	Return obj from Level folder and null if it doesn't exists.

Mod System	Description	Expected Result	Result
------------	-------------	-----------------	--------

1	GameStart Load Mods	Grabs Mods from save system and deploy Asset Bundle as a game object with its scripts.	Game Object spawns but scripts didn't come
---	---------------------	--	--

VALIDATION RESULTS

- Mod System works with asset bundles, but the asset bundles do not carry over scripts in current from meaning just assets are spawning
- UI Scroll wheels can form unintelligible results depending on camera view settings

CONCLUSION

The project, up to this point, consists of a main menu featuring a save system, a cloud integration system, and a semi-functional mod system.

- Save System: The system allows saving, loading, and deleting data in `Application.persistentDataPath`, with the option to place objects into a level-specific folder. This ensures that all game-related data is stored in a structured and organized manner and easy deletion.
- Cloud System: Mods are sent to and received from an Amazon S3 bucket, which serves as the central repository for all implemented mods. Saved in the format of `{Mod(bytes), Mod Name(string, key), Creator(string), Description(string), Photo(bytes)}`
- Mod System: The system utilizes Asset Bundles, which can be saved locally via the save system in the same format as the cloud system. Loaded at game start. Currently, scripts within the mods are not carried over, making the system partially functional but allowing for mod integration at runtime with the obj itself.

Overall, the project lays a solid foundation to work off of. Allowing game features to be added and the modification of implemented features.

Demo: [LastStandDemo.mp4](#)

PROJECT SUITABILITY

Mods:

Game: The game features are shown to not be complicated on their own sake but when put together it could be challenging time wise with a 3-group team with a limited time frame.

DOMAIN EXPERT EVALUATION

We don't have a domain expert yet.

USER TESTIMONIALS

- Add update functionality to the mod system.
- Implement improved coding practices, such as Lua scripting principles.
- Explore Addressables as an alternative to Asset Bundles.
- Conduct further research on mod items implementation.

FUTURE WORK

- Modify the save system by transitioning from Binary Formatter to Binary Writer for enhanced safety.
- Enhance the mod system to support updates for already downloaded mods and mods in a save file.
- Integrate scripting support when working with Unity objects in the mod system.
- Development and implementation of core game features.
- Refactor code to adhere to best practices and improve maintainability.
- Fix UI error with free camera mode.

GLOSSARY

- **Mods/Modding:** Modifying or adding new code to pre-existing code while not being linked to the creator of the original code.
- **Capstone:** The Project in which this game is a part of.
- **Oregon's Fall - The Final Stand:** title of our game
- **Cloud-based save data:** game data which is stored elsewhere accessible via the internet
- **Roguelike:** game mechanic where each playthrough is self-contained, with procedurally generated levels, permanent death, and little to no carryover between runs, making each attempt unique and independent from previous ones.

BIBLIOGRAPHY

- [1] Watt, J., & Allen, R. (2025). *A profile of the video game industry in the Canadian provinces*. Statistics Canada.
- [2] Steam. (2024). *Best of 2024*. <https://store.steampowered.com/charts/bestofyear/bestof2024>
- [3] SteamDB. (2025). *Steam charts*. <https://steamdb.info/charts/>
- [4] ConArtists. (2011, July 29). *The Last Stand - Union City* [Video game]. Armor Games. <https://armorgames.com/play/12009/the-last-stand-union-city>
- [5] ConArtists. (2008, April 26). *The Last Stand 2* [Video game]. Armor Games. <https://armorgames.com/play/1443/the-last-stand-2>
- [6] ConArtists. (2021, July 15). *The Last Stand Legacy Collection* [Video game]. Steam. https://store.steampowered.com/app/1615100/The_Last_Stand_Legacy_Collection/
- [7] Gameloft. (2022, November 14). *The Oregon Trail* [Video game]. Steam. https://store.steampowered.com/app/2013360/The_Oregon_Trail/

- [8] The Indie Stone. (2013, November 13). *Project Zomboid* [Video game]. Steam.
https://store.steampowered.com/app/108600/Project_Zomboid/
- [9] Entertainment Software Association. (2025). *America plays – and plays together* [Report].
<https://www.theesa.com/resources/essential-facts-about-the-us-video-game-industry/2025-data/>
- [10] Makuch, E. (2025, December 11). *Here's how much it could cost to show a game at The Game Awards*. GameSpot. <https://www.gamespot.com/articles/heres-how-much-it-could-cost-to-show-a-game-at-the-game-awards/1100-6536881/>
- [11] Clement, J. (2025, June 6). *Video gaming worldwide – statistics & facts*. Statista.
<https://www.statista.com/topics/1680/gaming/>
- [12] Duarte, F. (2025, November 20). *How many gamers are there? (New 2025 statistics)*. Exploding Topics.
<https://explodingtopics.com/blog/number-of-gamers>
- [13] Blair, N. (2024, December 31). *Video game industry statistics in Canada*. Made in CA. <https://madeinca.ca/video-game-industry-statistics-canada/>
- [14] Treyarch, Activision. (2015, November 6). *Call of Duty®: Black Ops III* [Video Game]. Steam.
https://store.steampowered.com/app/311210/Call_of_Duty_Black_Ops_III/